TP3 : écrire du JPEG avec la bibliothèque libjpeg

R3.L01 — Représentations et codages des images

Fiche d'explication du TP, écrite à partir du cours (CM diapos 55 à 74), du sujet du TP3, de `libjpeg.txt` et de `jpeg_example.c`.

Code complet et testé : `PROJET_FINAL/Image.cpp` (writeJPEG, readJPEG), `tp3.cpp`

Ce qu'il faut faire

La fonction demandée

```
void ColorImage::writeJPEG(const char* fname, unsigned int quality = 75) const;
```

libjpeg est écrite **en C** : elle écrit dans un `FILE*`, pas dans un `ostream`, d'où le nom de fichier en `const char*`.

D'abord sans `quality`, puis avec `jpeg_set_quality(&cinfo, quality, TRUE)` et la valeur par défaut 75 (dans le `.hpp` seulement).

1. Lire un PPM, tracer des rectangles, écrire en JPEG, ouvrir le résultat avec un visualiseur.
2. Écrire 21 fichiers `out_000.jpg` … `out_100.jpg` (qualité de 5 en 5) avec la boucle donnée par le sujet.
3. Comparer la qualité visuelle et la taille des fichiers.

Le sujet précise : **pas de gestionnaire d'erreur comme dans l'exemple C** (`setjmp`), on utilise les **exceptions** C++.

Compilation

Inclure l'en-tête C **dans un bloc `extern "C"`** :

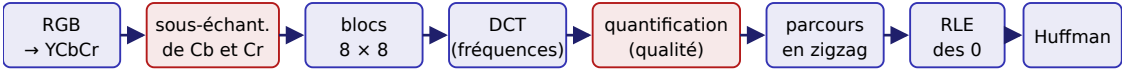
```
#include <cstdio>   // FILE, avant jpeglib.h
extern "C" {
#include <jpeglib.h>
}
```

Et lier avec la bibliothèque :

```
g++ -Wall -c programme.cpp
g++ -o programme programme.o -ljpeg
```

Debian/Ubuntu : `apt install libjpeg-dev`

Comment JPEG compresse CM diapos 66 et 67



En rouge : les deux étapes où l'on PERD de l'information (c'est voulu).

Toutes les autres étapes sont réversibles (ou presque) : RLE et Huffman sont des compressions sans perte.

Le paramètre « qualité » règle la quantification : plus il est bas, plus on arrondit fort, plus il y a de 0.

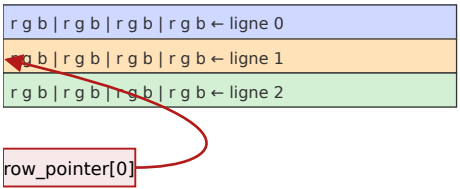
- **YCbCr** : on sépare la luminance Y (le noir et blanc) des deux chrominances. L'œil voit mieux les formes en luminance qu'en couleur (CM diapo 32).
- **Sous-échantillonnage** : on garde moins de points de couleur (souvent 1 pour 2 × 2 pixels).
- **DCT** sur chaque bloc 8 × 8 : on passe des pixels à des coefficients de fréquence. Les détails fins (hautes fréquences) sont peu visibles : on les arrondit fort.
- Après quantification, beaucoup de coefficients valent 0 : **RLE** des zéros avec une marque de fin de bloc, puis **Huffman**.

writeJPEG : les étapes de libjpeg.txt

```
void ColorImage::writeJPEG(const char* fname, unsigned int
quality) const
{
  if (quality > 100)
    throw std::invalid_argument("writeJPEG : qualité hors de
[0;100]");
  FILE* outfile = std::fopen(fname, "wb"); // "b" : binaire !
  if (outfile == nullptr)
    throw std::runtime_error(std::string("writeJPEG : impossible
de créer ") + fname);
  // 1. Allocation et initialisation de l'objet de compression
  jpeg_compress_struct cinfo;
  jpeg_error_mgr       jerr;
  cinfo.err = jpeg_std_error(&jerr);
  jerr.error_exit = jpeg_error_to_exception; // voir plus loin
  jpeg_create_compress(&cinfo);
  try {
    // 2. Destination des données compressées
    jpeg_stdio_dest(&cinfo, outfile);
    // 3. Description de l'image et paramètres
    cinfo.image_width      = width;
    cinfo.image_height     = height;
    cinfo.input_components = 3;          // r, g, b
    cinfo.in_color_space   = JCS_RGB;
    jpeg_set_defaults(&cinfo);         // APRÈS in_color_space !
    jpeg_set_quality(&cinfo, int(quality), TRUE);
    // 4. Compression, une ligne (scanline) à la
fois
    jpeg_start_compress(&cinfo, TRUE);
    while (cinfo.next_scanline < cinfo.image_height)
{
      JSAMPROW row_pointer[1];  // pointeur vers une
ligne
      row_pointer[0] = (JSAMPROW)(array
                + size_t(cinfo.next_scanline) *
width);
      jpeg_write_scanlines(&cinfo, row_pointer, 1);
    }
    // 5. Fin de la compression
    jpeg_finish_compress(&cinfo);
  } catch (...) {     // erreur : on libère puis on
relance
    jpeg_destroy_compress(&cinfo);
    std::fclose(outfile);
    throw;
  }
  // 6. Libération de l'objet et fermeture du
fichier
  jpeg_destroy_compress(&cinfo);
  std::fclose(outfile);
}
```

Donner les lignes de pixels à libjpeg

array (ColorImage, width = 4)



array + next_scanline × width (ici next_scanline = 1)

libjpeg attend, pour chaque ligne, un pointeur vers **3 × width octets** r, g, b à la suite : `JSAMPROW` = `unsigned char*`.

Notre tableau de `Color` a exactement cette forme (3 octets par `Color`, sans remplissage, CM diapo 14). Il suffit donc de pointer sur le début de la ligne :

```
row_pointer[0] = (JSAMPROW)(array +
size_t(cinfo.next_scanline) * width);
```

- `array + n` sur un `Color*` avance de `n` **Color**, soit 3n octets.
- Le cast `(JSAMPROW)` enlève le `const` : sans danger, la compression ne modifie pas les pixels.
- On envoie les lignes de haut en bas, l'ordre attendu par JPEG.

Les erreurs de libjpeg deviennent des exceptions

En cas d'erreur grave (fichier qui n'est pas un JPEG, disque plein...), libjpeg appelle `cinfo.err->error_exit`. Celle par défaut **affiche un message puis appelle `exit()`** : le programme s'arrête net, l'appelant ne peut pas réagir.

On la remplace par une fonction à nous qui lève une exception :

```
static void jpeg_error_to_exception(j_common_ptr cinfo)
{
  char message[JMSG_LENGTH_MAX];
  (*cinfo->err->format_message)(cinfo, message);
  throw std::runtime_error(std::string("libjpeg : ") +
message);
}
...
cinfo.err = jpeg_std_error(&jerr);
jerr.error_exit = jpeg_error_to_exception;
```

Le try / catch(...) / throw;

Si une exception part en pleine compression, il faut quand même **détruire l'objet libjpeg et fermer le fichier**. On attrape tout (`catch (...)`), on libère, puis `throw;` relance **la même** exception vers l'appelant.

Testé dans tp3.cpp

```
try {
  ColorImage* p =
ColorImage::readJPEG("images/chat.ppm");
  delete p;
} catch (const std::runtime_error& e) {
  std::cout << e.what() << std::endl;
}
```

affiche : `libjpeg : Not a JPEG file: starts with 0x50 0x36` (0x50 0x36 = "P6" !).

Qualité contre taille (chat 320 × 240 avec les 3 cadres)

fichier	taille	taux
tp3_chat.ppm (brut)	230 483 o	1 : 1
out_100.jpg	73 468 o	3 : 1
out_090.jpg	29 368 o	8 : 1
out_075.jpg = défaut	18 908 o	12 : 1
out_050.jpg	13 334 o	17 : 1
out_025.jpg	9 358 o	25 : 1
out_005.jpg	3 955 o	58 : 1
out_000.jpg	3 005 o	77 : 1

taux = taille originale / taille compressée (CM diapo 55).
`tp3_chat.jpg` (sans qualité donnée) fait aussi 18 908 octets : le défaut est bien 75.



PPM



100



75



50



25



5

Même à 100 %, les cadres sont plus ternes et baveux (sujet) : la couleur est sous-échantillonnée, et un trait d'1 pixel est un détail très fin, le pire cas pour JPEG.

Bonus : readJPEG

```
ColorImage* ColorImage::readJPEG(const char* fname)
{
  FILE* infile = std::fopen(fname, "rb");
  if (infile == nullptr)
    throw std::runtime_error(std::string("readJPEG :
impossible d'ouvrir ") + fname);
  jpeg_decompress_struct cinfo;
  jpeg_error_mgr         jerr;
  cinfo.err = jpeg_std_error(&jerr);
  jerr.error_exit = jpeg_error_to_exception;
  jpeg_create_decompress(&cinfo);
  ColorImage* res = nullptr;
  try {
    jpeg_stdio_src(&cinfo, infile);
    jpeg_read_header(&cinfo, TRUE);  // taille,
composantes...
    cinfo.out_color_space = JCS_RGB;  // toujours r, g, b
    jpeg_start_decompress(&cinfo);
    if (cinfo.output_width > 65535 || cinfo.output_height >
65535)
      throw std::runtime_error("readJPEG : image trop
grande");
    res = new ColorImage(uint16_t(cinfo.output_width),
                         uint16_t(cinfo.output_height));
    while (cinfo.output_scanline < cinfo.output_height) {
      JSAMPROW row_pointer[1];
      row_pointer[0] = (JSAMPROW)(res->array
            + size_t(cinfo.output_scanline) * res->width);
      jpeg_read_scanlines(&cinfo, row_pointer, 1);
    }
    jpeg_finish_decompress(&cinfo);
  } catch (...) {
    delete res;           // delete nullptr ne fait rien
    jpeg_destroy_decompress(&cinfo);
    std::fclose(infile);
    throw;
  }
  jpeg_destroy_decompress(&cinfo);
  std::fclose(infile);
  return res;
}
```

- Même structure à l'envers. La taille n'est connue qu'après `jpeg_read_header` : on crée l'image à ce moment-là.
- Test : dans `out_075.jpg`, le pixel (10, 120) du cadre vert (0, 255, 0) vaut **(89, 189, 93)** : c'est la perte JPEG.

Ce qui a été corrigé, et questions possibles

Version Gemini	Correction
Gestionnaire d'erreur par défaut : la moindre erreur libjpeg arrête tout le programme avec `exit()`.	`error_exit` remplacé par une fonction qui lève une exception (demandé par le sujet) + libération dans un `catch`.
Qualité non vérifiée.	`invalid_argument` si > 100.
GrayImage absente du dossier TP3.	Un seul projet avec tout.
Cadres à 10, 14 et 18 pixels.	5, 10 et 15 pixels comme sur la figure du sujet.

1. **Pourquoi `extern "C"` ?** Les fonctions sont compilées en C : sans ce bloc, le compilateur C++ chercherait des noms « décorés » (surcharge) et l'édition de liens échouerait.
2. **À quoi sert `-ljpeg` ?** À lier le programme avec la bibliothèque, à l'édition de liens.
3. **JPEG est-il adapté à un schéma, un logo ?** Non (CM diapo 66) : contours nets et peu de couleurs → PNG ou GIF.
4. **Où est la perte ?** Sous-échantillonnage des couleurs et quantification des coefficients de la DCT.
5. **Taux de compression de 230 Ko vers 19 Ko ?** $\approx 12 : 1$.